

Physical Clocks

Problem

Sometimes we need the **exact time**, not only the order of events.

Solution: UTC (Universal Coordinated Time)

UTC is used as a common real time standard.

- It is based on cesium atomic clocks.
- The real time is calculated from many cesium clocks around the world.
- Leap seconds can be added because days slowly become longer.

Note

UTC is broadcast by radio and satellite.
Satellites can provide very accurate time.

Clock Synchronization

Precision

Precision means keeping the difference between clocks small.

The goal is:

clocks on different machines should stay close to each other.

Accuracy

Accuracy means keeping a clock close to the real UTC time.

The goal is:

a machine's clock should be close to the real time.

Synchronization

- **Internal synchronization:** keeps clocks close to each other. (**precise**)
- **External synchronization:** keeps clocks close to real UTC time. (**accurate**)

Clock Specifications

A clock has a maximum **clock drift rate p**.

This means a clock can run a little faster or slower than the ideal clock.

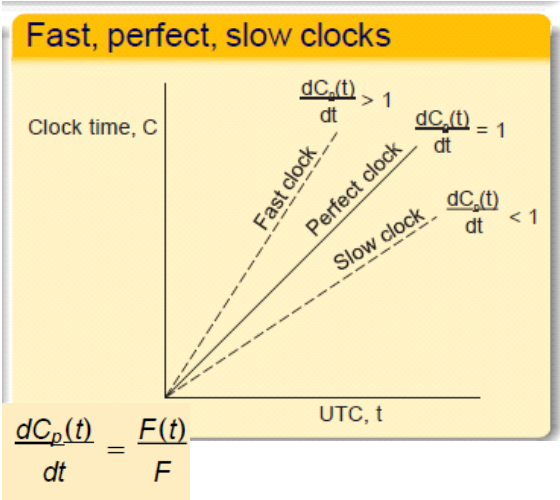
- **F(t)** is the real oscillator frequency at time **t**.
- **F** is the ideal constant frequency.
- A clock can be **fast, perfect, or slow**.

Main Idea

Real clocks are not perfect.

Their time can slowly move away from the correct time.

This is called **clock drift**.



Detecting and Adjusting Incorrect Times

Getting Time from a Time Server

A machine can ask a time server for the current time.

- **T1:** client sends request.
- **T2:** server receives request.
- **T3:** server sends response.
- **T4:** client receives response.

Offset and Delay

The system estimates:

- **θ (offset):** how much the client clock differs from the server clock.
- **δ (delay):** how long the message takes in the network. (network lag)

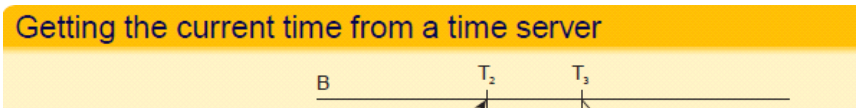
Network Time Protocol

NTP collects several **θ** and **δ** pairs.
Then it chooses the **θ** value with the smallest **δ**.

Main Idea

The smallest delay usually **gives the best time estimate**.

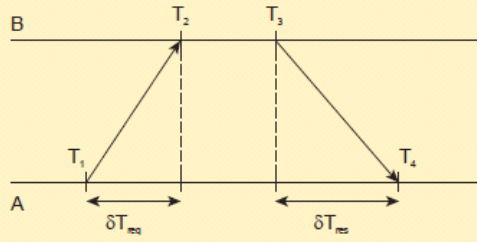
So, the client can adjust its clock more accurately.



$$\delta = (T_4 - T_1) - (T_3 - T_2) / 2$$

Gidiş süresi ≈ dönüş süresi
T4 - T1 = gidiş delay + server işlem süresi + dönüş delay

Getting the current time from a time server



Gidiş süresi $\approx$ dönüş süresi

$T_4 - T_1$ = gidiş delay + server işlem süresi + dönüş delay

$T_3 - T_2$ = Server işlem süresi

$$\theta = T_3 + (T_2 - T_1) + (T_4 - T_3) / 2 - T_4 = (T_2 - T_1) + (T_3 - T_4) / 2$$

Gidiş yönünden saat farkını tahmin et, dönüş yönünden saat farkını tahmin et, ikisinin ortalamasını al.

Keeping Time Without UTC

Principle

A time server can check all machines periodically.

It calculates an average time and tells each machine how to adjust its clock.

Using a Time Server

Machines do not directly jump to the new time.

They adjust their clocks smoothly by running faster or slower.

Fundamental Rule

Setting the clock backward is not allowed.

So, time corrections should be done with smooth adjustments.

The Happened-Before Relationship

Issue

In distributed systems, the exact time is not always the most important thing.

Usually, the important thing is the order of events.

Happened-Before Rules

- If event **a** happens before event **b** in the same process, then $a \rightarrow b$.
- If **a** is sending a message and **b** is receiving that message, then $a \rightarrow b$.
- If $a \rightarrow b$ and $b \rightarrow c$, then $a \rightarrow c$.

Logical Clocks

Problem

We need a global view of the system that follows the happened-before relation.

Timestamp Rule

Each event **e** gets a timestamp $C(e)$.

- **P1:** If event **a** happens before event **b** in the same process, then $C(a) < C(b)$.
- **P2:** If event **a** is sending a message and event **b** is receiving it, then $C(a) < C(b)$.

Main Problem

There is no global clock.

So, each process must keep its own logical clock.

Logical Clocks: Solution

Local Counter

Each process P_i keeps a local counter C_i .

1. For every new event in P_i , increase C_i by 1.
2. When P_i sends a message, the message gets timestamp $ts(m) = C_i$.
3. When P_j receives a message, it sets its counter to:

$C_j = \max(C_j, ts(m))$

Then it increases the counter by 1 before processing the message.

Notes

- Rule 1 keeps events ordered inside the same process.
- Rules 2 and 3 keep send events before receive events.
- If two events get the same timestamp, process IDs can be used to break the tie.

Local counter doesn't show us the real time, it shows the order of the events.

P1 mesaj gönderiyor:

- P1 clock = 5
- Mesaj timestamp $C1 = 5$

P2'nin clock'u o anda 3 olsun.

P2 mesajı alınca:

$\max(3, ts(5)) = 5$

Sonra +1 yapar:

P2 clock = 6

Böylece:

- send event = 5
- receive event = 6

Logical Clocks: Where Implemented Main Idea

Logical clock adjustments are implemented in the **middleware layer**.

When an application **sends a message**, middleware:

- increases the local clock
- adds a timestamp to the message
- sends the message through the network

When a **message is received**, middleware:

- reads the timestamp
- adjusts the local clock
- then delivers the message to the application

Example: Total-Ordered Multicast

Problem

Concurrent updates on a replicated database must be seen in the **same order** everywhere.

Example:

- **P1** adds **\$100** to an account.
- **P2** increases the account by **1%**.
- There are two database replicas.

If replicas apply updates in different orders, they may get different results.

Total-Ordered Multicast: Solution

Main Idea LAMPORT CLOCKS

The goal is that all replicas process updates in the **same order**.

Each process sends a message with a **timestamp** to all other processes.

Incoming messages are stored in a **local queue and ordered by timestamp**.

Delivery Rule

A message is not delivered to the application immediately.

It waits in the queue until the system is sure that no older message can arrive later.

A process **delivers** message **mi** only if:

1. **mi** is at the head of the local queue.
2. Did process has seen larger timestamped messages from all other processes ?

This guarantees that every replica applies updates in the same order.

Lamport's Clocks for Mutual Exclusion

Main Idea

This is similar to **total-ordered multicast**.

In total-ordered multicast, all processes keep the same queue and deliver messages in the same order.

In mutual exclusion, processes must agree on the order for entering a **critical section**.

Note : Lamport clocks can help decide which process **enters the critical section first**.

aynı dosyaya işlem için yarısan iki processten timestampi küçük olan critical sectiona girer ve ilk işlemi yapar .

Vector Clocks

Problem

Lamport clocks can show event ordering, but they **do not always show causality**.

If **C(a) < C(b)**, this does not always mean that **a caused b**.

- Event **a** happens at logical time **16**.
- Event **b** happens at logical time **20**.

Even if **16 < 20**, we cannot say that **a causally precedes b**.

Short Meaning : Lamport clocks are not enough to detect real causal relationships.

For this, we need **vector clocks**.

önce olmus olması a nın b ye neden olduğu (casuality) anlamına gelmez , bunun için lambort's clock yeterli değildir, vector clock kullanmamız gerekir.

Causal Dependency

Main Idea

If event **a** causally precedes event **b**, then **b** may depend on **a**.
This means information from **a** may be carried to **b**.

Shortly : Event **b** can be affected by event **a**.
So, $a \rightarrow b$ shows a possible causal dependency.

Vector clocks are the **solution for the limitation of Lamport clocks**.

Lamport clocks can say:

$C(a) < C(b)$

But this does **not always mean** that event **a** causally affected **b**.

So, we use **vector clocks** to understand real causal dependency between events.

Capturing Causality with Vector Clocks

Short Logic

A **vector clock** is a list of event counters.

Each process keeps information about:

- its own events
- events it knows from other processes



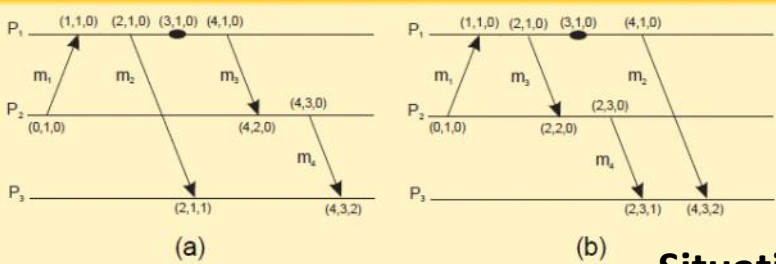
Rules

LİSTE VAR !

- When a process has its own event, it **increases** its own index.
- When a process sends a message, it attaches its vector clock to the message.
- When a process receives a message, it takes the maximum values between its own vector and the received vector.
- Then it increases its own index for the receive event.

Vector clocks: Example

Capturing potential causality when exchanging messages



Situation (a)

In the table:

$ts(m2) = (2, 1, 0)$

$ts(m4) = (4, 3, 0)$

Here:

$m2 < m4$

Because all vector values of **m2** are smaller than or equal to the values of **m4**.

So, the conclusion is:

m2 may causally precede m4.

This means **m2 may have happened before m4**, and information from **m2** may have affected **m4**.

Situation (b)

In the table:

$ts(m2) = (4, 1, 0)$

$ts(m4) = (2, 3, 0)$

Comparison:

- First index: $4 > 2$
- Second index: $1 < 3$
- Third index: $0 = 0$

Here, one value is larger and one value is smaller.

So, we cannot say: **m2 < m4**

and we also cannot say: **m2 > m4**

There is no clear causal order between these two messages.

So, the conclusion is:

m2 and m4 may conflict.

This means they may be **concurrent** or independent events.

Mutual Exclusion

Problem

Many processes may want exclusive access to the **same resource**.
Only one process should enter the **critical section** at a time.

Basic Solutions

Permission-based:

A process must get permission from other processes before entering the critical section.

Token-based:

A token is passed between processes.
Only the process with the token can enter the critical section.

Short Meaning

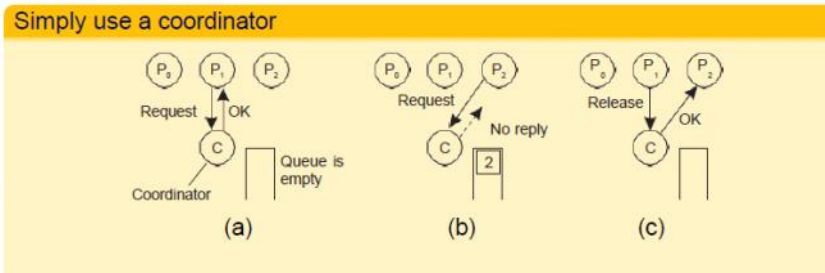
Mutual exclusion controls access to shared resources.
It prevents multiple processes from using the same critical section at the same time.

1- Permission-Based, Centralized

A **coordinator** controls access to the shared resource.

Steps

- (a) Process **P1** asks the coordinator for permission.
The coordinator gives **OK**, so **P1** enters critical section.
- (b) Process **P2** also asks for permission.
But **P1** is still using the resource, so the coordinator does not reply yet.
P2 waits in the queue.
- (c) When **P1** releases the resource, it tells the coordinator.
Then the coordinator sends **OK** to **P2**.



Mutual Exclusion: 2- Ricart & Agrawala

Main Idea

Ricart & Agrawala is a **distributed mutual exclusion** algorithm.
There is no central coordinator.

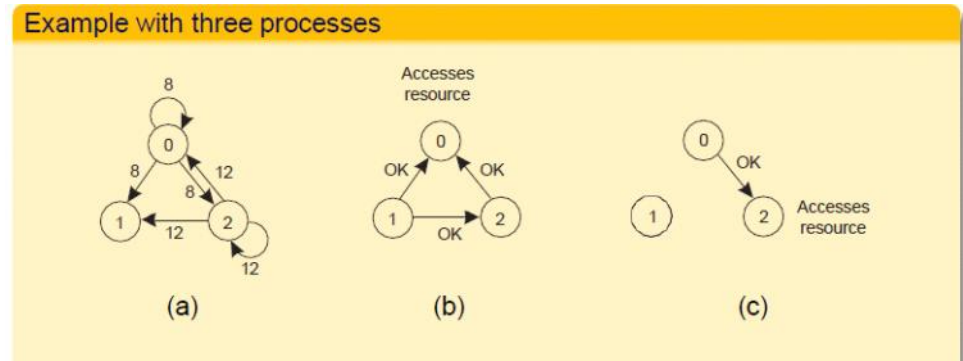
A process replies to a request only when:

- it **does not want the shared resource**, or
- it also wants the resource, but the requester has **higher priority**.

Priority is decided by comparing **timestamps**.

Deferred Reply

If the receiving process has higher priority, it does not reply immediately.
The reply is **deferred** until later.



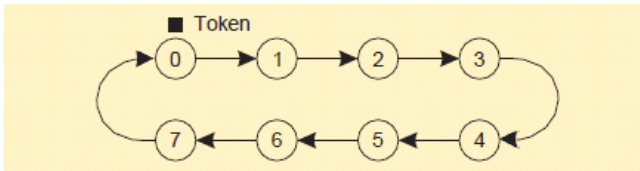
Three Processes

Two processes want to access the shared resource at the same time.

- **P0** has timestamp **8**.
- **P2** has timestamp **12**.

Since **P0** has the smaller timestamp, **P0** has higher priority and **enters** the critical section first.

When **P0** finishes, it sends **OK** to **P2**.
Then **P2** can access the resource.



Mutual Exclusion: 3- Token Ring Algorithm

Main Idea

Processes are organized in a **logical ring**.

A **token** is passed between processes.

Only the process that holds the token can enter the **critical region**.

If a process wants to enter the critical section, it **waits** for the token.

If it gets the token, it can **enter**.

If it does not need the resource, it **passes** the token to the next process.

4- Decentralized Mutual Exclusion

Principle

The resource is replicated **N times**.

Each replica has its own coordinator.

To access the resource, a process needs permission from a **majority** of coordinators:

$$m > N / 2$$

So, one coordinator is not enough.

The **process must get more than half of the permissions**.

Assumption

If a coordinator crashes, it can recover quickly.

But after recovery, it may **forget** the permissions it gave before.

Short Meaning

There is no single central coordinator.

A process enters the critical section only if it gets permission from the majority.

Election Algorithms

Principle

Some algorithms need one process to act as a **coordinator**.

The problem is **how to choose this coordinator dynamically**.

Note

In some systems, the coordinator is chosen manually.

But this creates a centralized solution and may cause a **single point of failure**.

Basic Assumptions

- Every process has a unique ID.
- Every process knows the IDs of all other processes.
- But a process may not know which processes are currently up or down.

Election Meaning

Election means choosing the process with the **highest ID** that is still running.

Election by Bullying

Principle

When a process notices that the coordinator is not responding, it starts an election.

Process **P_k** sends an **ELECTION** message to all processes with higher IDs.

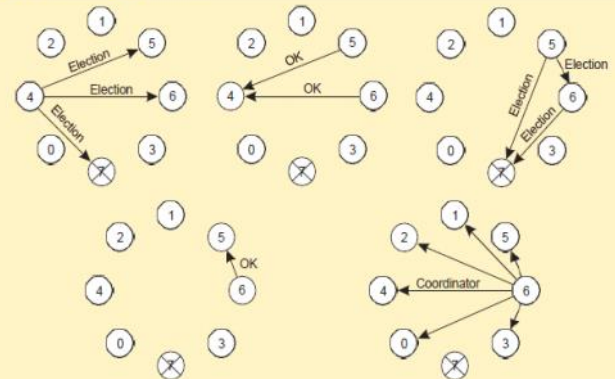
- If no higher process replies, **P_k** wins and becomes the coordinator.
- If a higher process replies, that higher process takes over the election.
- Finally, the process with the highest active ID becomes the coordinator.

Example

- Process **4** notices that the coordinator is down.
- It sends **ELECTION** messages to higher-ID processes.
- Higher processes **5** and **6** reply with **OK**.
- So, process **4** stops its election.
- Then process **6** continues the election.

Since **6** has the highest active ID, it becomes the new coordinator.

The bully election algorithm



Election in a Ring

Principle

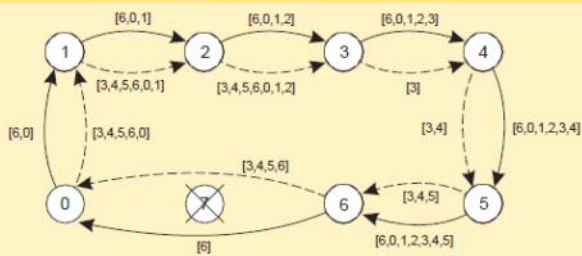
Processes are organized in a **logical ring**.

The process with the highest priority becomes the coordinator.

Any process can start an election by sending an **election message** to its successor.

- If a successor is down, the message goes to the next successor.
- Each active process adds itself to the list.
- When the message returns to the initiator, all active processes are known.
- Then a coordinator message is sent around the ring.
- The active process with the highest priority is elected as coordinator.

Election algorithm using a ring

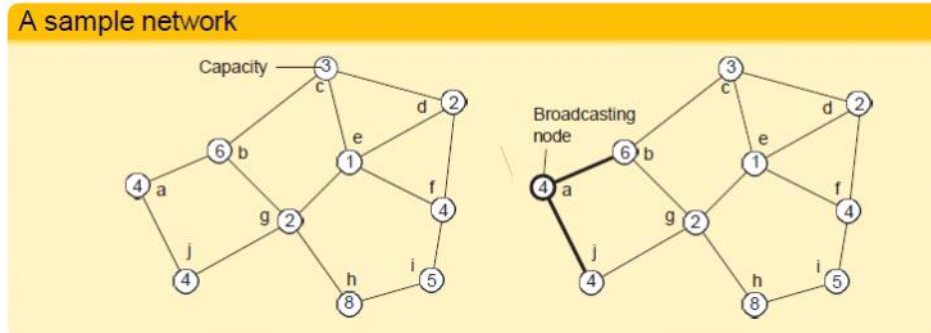


- The solid line shows the election messages initiated by P_6
- The dashed one the messages by P_3

An election message moves around the ring and collects active process IDs.

In the example:

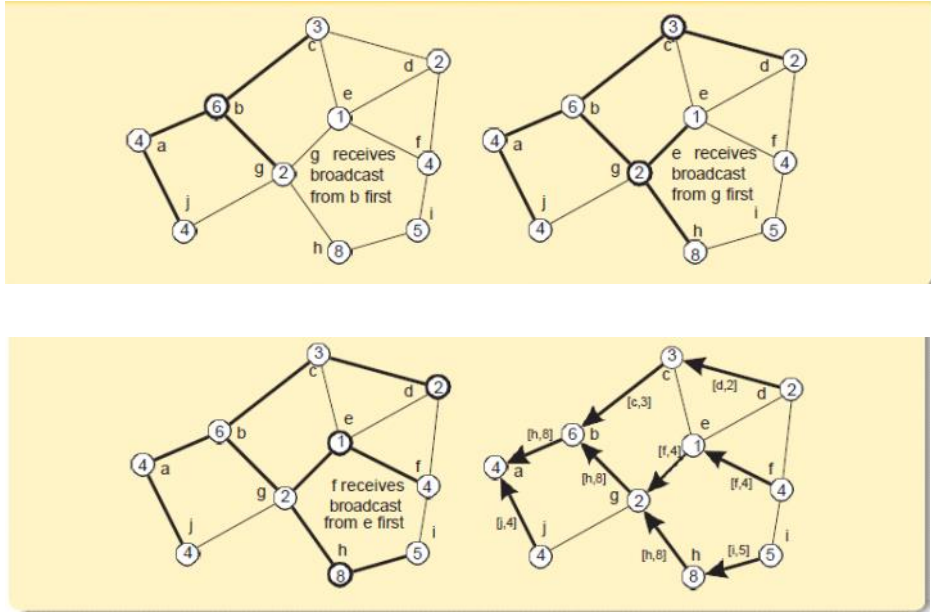
- The solid line shows the election started by **P6**.
- The dashed line shows the election started by **P3**.
- Failed processes are skipped.
- At the end, the active process with the highest ID becomes the coordinator.



In wireless networks, nodes may not reach all other nodes directly.
So, election messages are spread by **broadcasting**.
Each node has a **capacity value**.
The node with the highest suitable capacity can become the coordinator.

Broadcast Tree

When a node receives a broadcast for the first time, it accepts that path.
This creates a **broadcast tree** shown with thick lines.
The message spreads through the network and collects information from active nodes.



Coordinator Selection

After the broadcast finishes, the system can compare capacity values.
The node with the highest capacity is selected as coordinator.